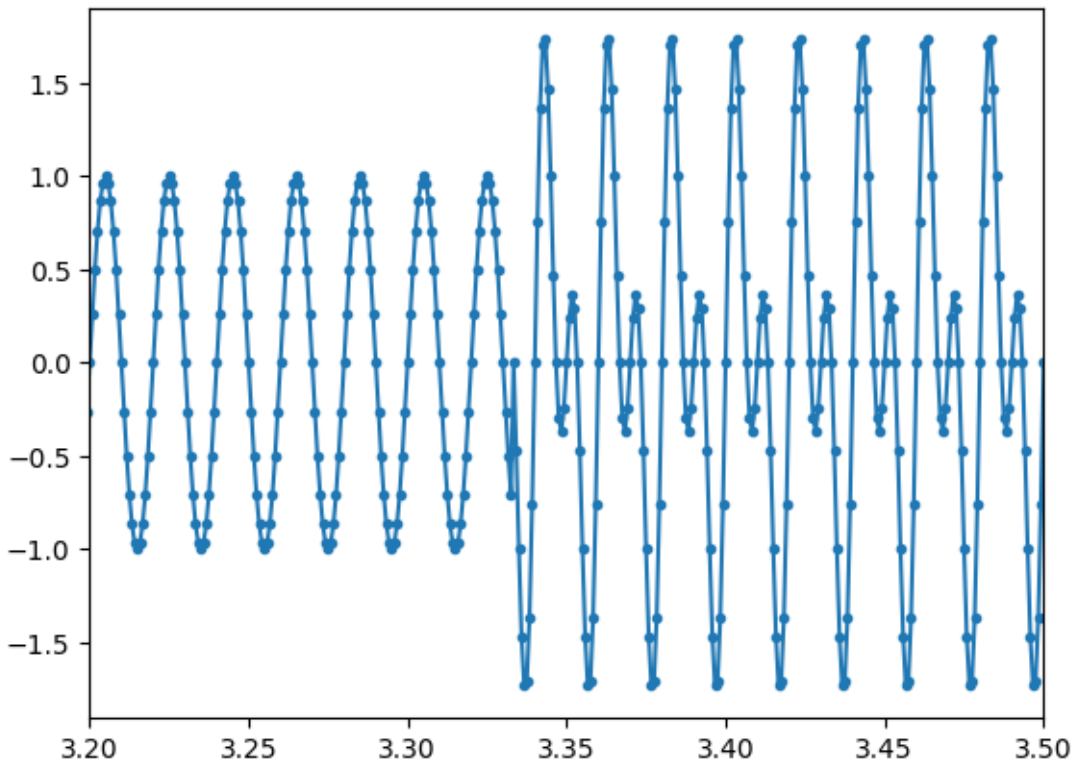
lab13

May 31, 2026

0.1 Continuous wavelets

[182]:
```
# Create a Python script to analyze a time series using the Continuous Wavelet␣
↪Transform (CWT) and plot the time-frequency representation.
import numpy as np
import matplotlib.pyplot as plt
import pywt

# Generate a sample time series
t = np.linspace(0, 10, 12000, endpoint=False)
fs = 1/(t[1] - t[0])
x = np.zeros(len(t))
x[:len(x)//3*2] += np.sin(2 * np.pi * 50  * t[:len(t)//3*2])
x[len(x)//3:  ] += np.sin(2 * np.pi * 100 * t[len(t)//3:])
plt.plot(t, x, '.-')
plt.xlim([3.2, 3.5])
```

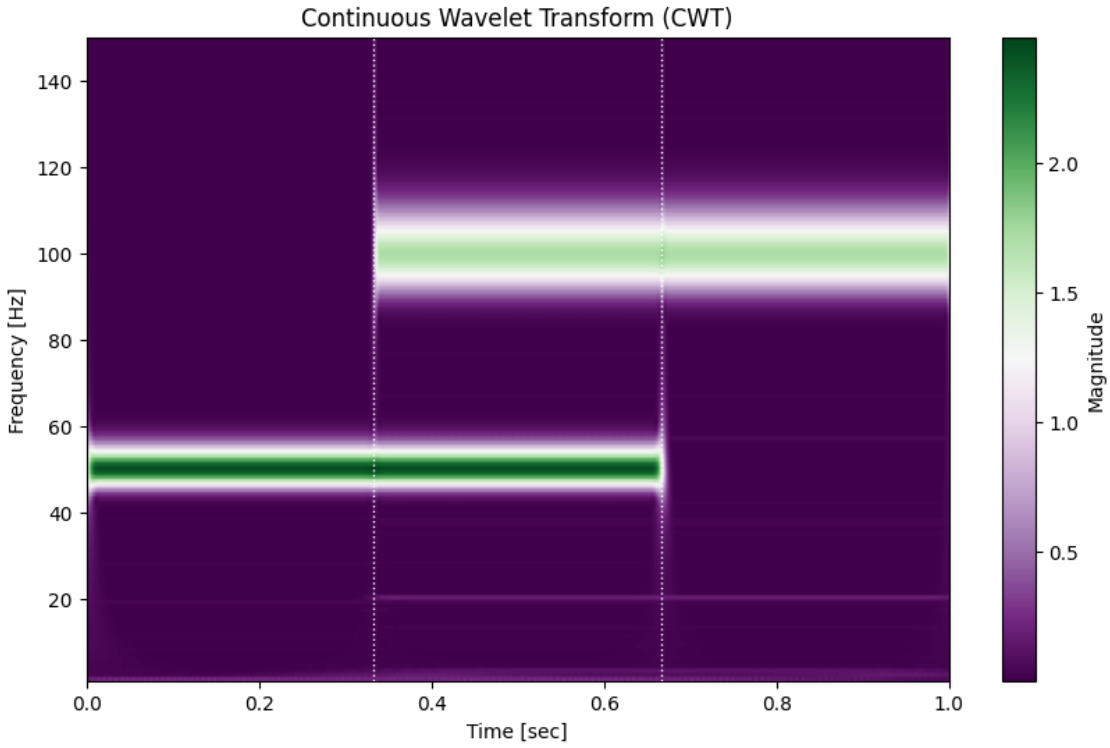
[182]:
```
(3.2, 3.5)
```



[183]:
```
# Define the wavelet and the desired frequency range
wavelet = 'cmor12-1.0'  # Specify the bandwidth frequency and center frequency
#wavelet = 'morl'  # Specify the bandwidth frequency and center frequency

# Calculate scales to match the desired frequencies
min_freq = 1  # Minimum frequency of interest in Hz
max_freq = 150  # Maximum frequency of interest in Hz
frequencies = np.linspace(min_freq, max_freq, num=150)
scales = pywt.central_frequency(wavelet) * fs / frequencies
print(pywt.central_frequency(wavelet))
print(scales)
# Compute the CWT using the specified wavelet
coefficients, _ = pywt.cwt(x, scales, wavelet, sampling_period=1/fs)

# Plot the time-frequency representation
plt.figure(figsize=(10, 6))
plt.imshow(np.abs(coefficients), extent=[0, 1, max_freq, min_freq],␣
 ↪cmap='PRGn', aspect='auto')
plt.axvline(1/3, color='white', linewidth=1, linestyle=':')
plt.axvline(2/3, color='white', linewidth=1, linestyle=':')
plt.title('Continuous Wavelet Transform (CWT)')
plt.ylabel('Frequency [Hz]')
```

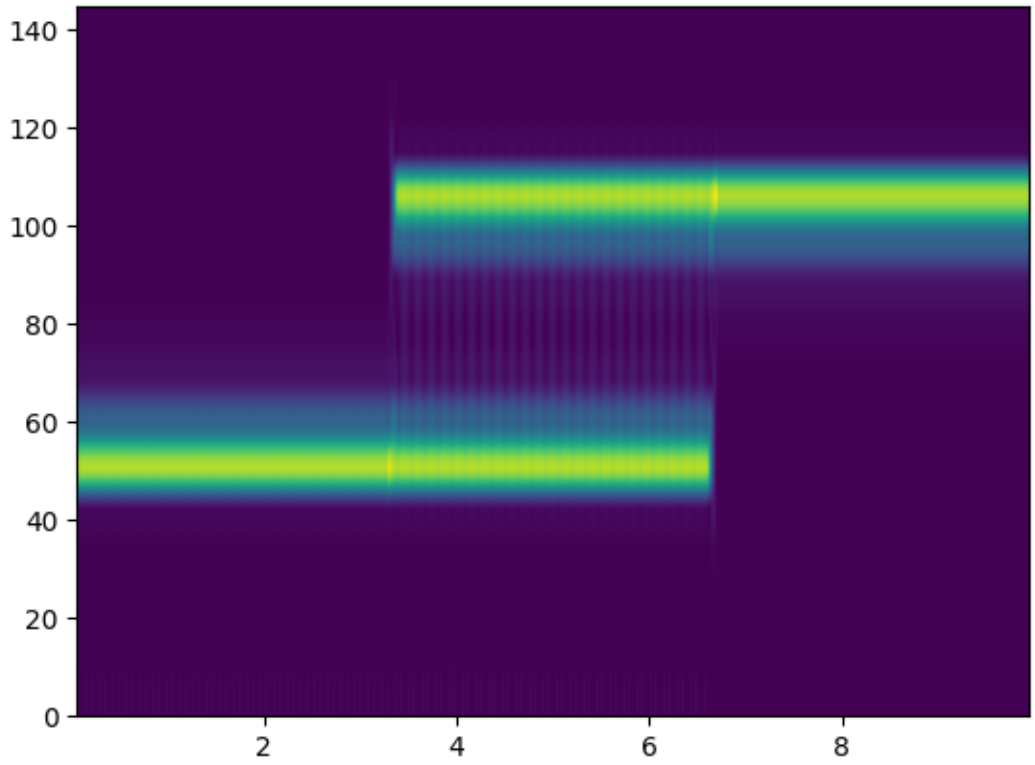
```
plt.xlabel('Time [sec]')
plt.colorbar(label='Magnitude')
plt.ylim(max_freq, min_freq)  # Ensure correct y-axis limits
plt.gca().invert_yaxis()  # Invert y-axis to have low frequencies at the bottom
plt.show()
```

```
1.0
[1200.          600.          400.          300.          240.
  200.          171.42857143  150.          133.33333333  120.
  109.09090909  100.           92.30769231   85.71428571   80.
   75.           70.58823529   66.66666667   63.15789474   60.
   57.14285714   54.54545455   52.17391304   50.           48.
   46.15384615   44.44444444   42.85714286   41.37931034   40.
   38.70967742   37.5          36.36363636   35.29411765   34.28571429
   33.33333333   32.43243243   31.57894737   30.76923077   30.
   29.26829268   28.57142857   27.90697674   27.27272727   26.66666667
   26.08695652   25.53191489   25.           24.48979592   24.
   23.52941176   23.07692308   22.64150943   22.22222222   21.81818182
   21.42857143   21.05263158   20.68965517   20.33898305   20.
   19.67213115   19.35483871   19.04761905   18.75         18.46153846
   18.18181818   17.91044776   17.64705882   17.39130435   17.14285714
   16.90140845   16.66666667   16.43835616   16.21621622   16.
   15.78947368   15.58441558   15.38461538   15.18987342   15.
   14.81481481   14.63414634   14.45783133   14.28571429   14.11764706
   13.95348837   13.79310345   13.63636364   13.48314607   13.33333333
   13.18681319   13.04347826   12.90322581   12.76595745   12.63157895
   12.5          12.37113402   12.24489796   12.12121212   12.
   11.88118812   11.76470588   11.65048544   11.53846154   11.42857143
   11.32075472   11.21495327   11.11111111   11.00917431   10.90909091
   10.81081081   10.71428571   10.61946903   10.52631579   10.43478261
   10.34482759   10.25641026   10.16949153   10.08403361   10.
    9.91735537    9.83606557    9.75609756    9.67741935    9.6
    9.52380952    9.4488189     9.375         9.30232558    9.23076923
    9.16030534    9.09090909    9.02255639    8.95522388    8.88888889
    8.82352941    8.75912409    8.69565217    8.63309353    8.57142857
    8.5106383     8.45070423    8.39160839    8.33333333    8.27586207
    8.21917808    8.16326531    8.10810811    8.05369128    8.        ]
```



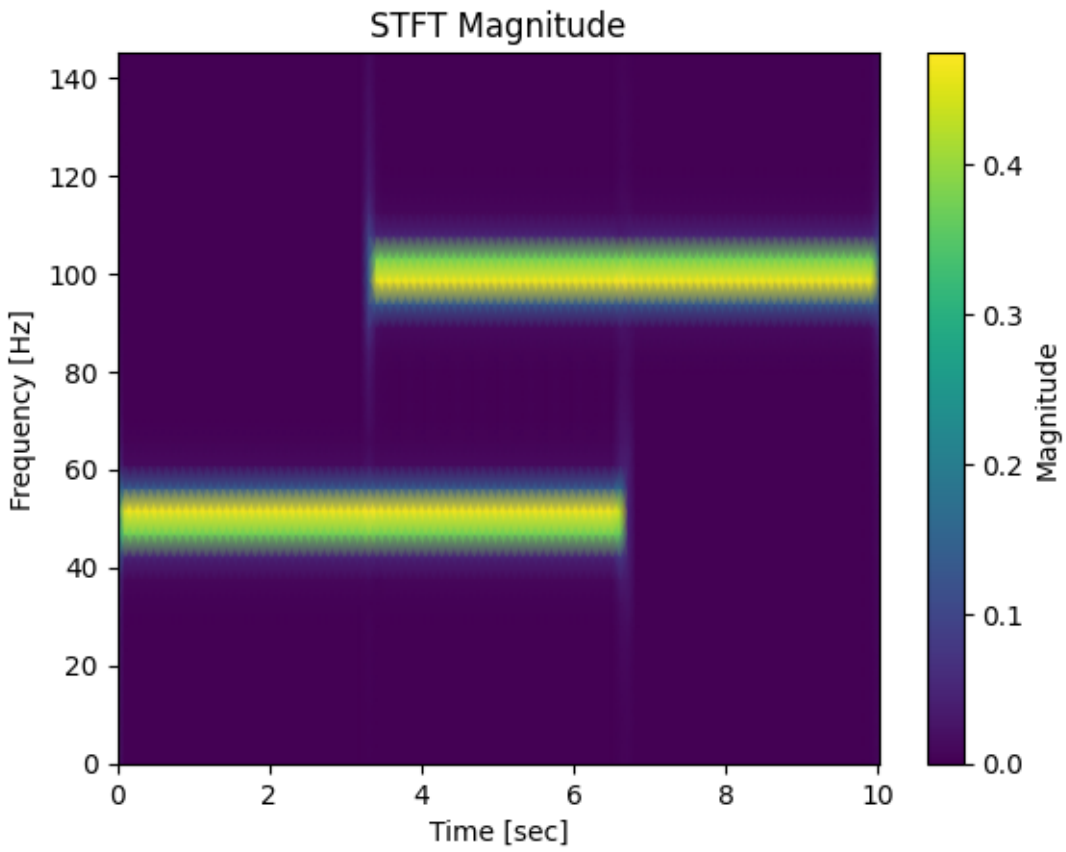
[184]:
```
# Implement a Python script to compute and plot the Short Time Fourier␣
 ↪Transform (STFT) of a given signal.
# Method 1: using spectrogram
from pylab import *
from scipy import signal
from numpy import random
f, t, s = signal.spectrogram(x, fs=sampling_frequency, nperseg=128,␣
 ↪noverlap=100) # better resolution
imshow(s, extent=[t.min(), t.max(), f.min(), f.max()], aspect='auto',␣
 ↪origin='lower')
ylim([0, 145])
```

[184]:
```
(0.0, 145.0)
```



[185]:
```
# Method 2: using stft
from scipy.signal import stft

# Compute the STFT
f, t, Zxx = stft(x, fs, nperseg=256)

# Plot the STFT magnitude
plt.pcolormesh(t, f, np.abs(Zxx), shading='gouraud')
plt.title('STFT Magnitude')
plt.ylabel('Frequency [Hz]')
plt.xlabel('Time [sec]')
plt.colorbar(label='Magnitude')
plt.ylim([0, 145])
```

[185]: (0.0, 145.0)



[186]:
```
pywt.wavelist()
```

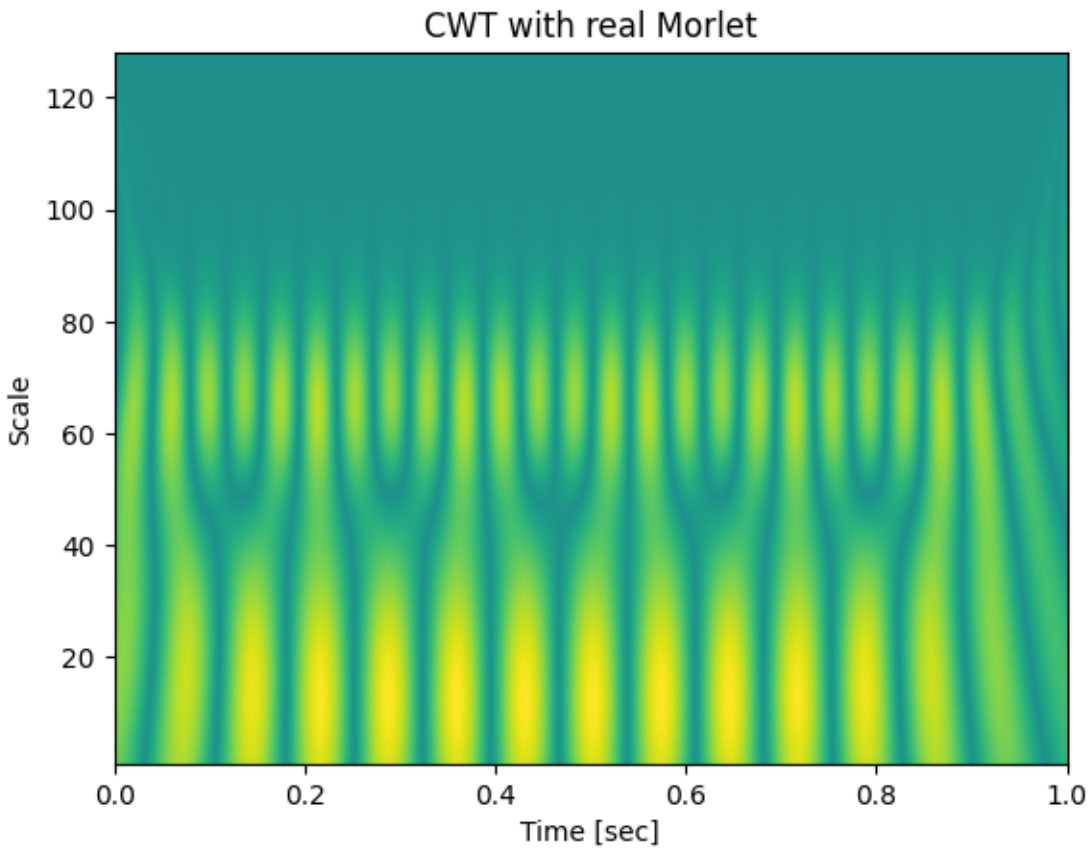
[186]:
```
['bior1.1',
 'bior1.3',
 'bior1.5',
 'bior2.2',
 'bior2.4',
 'bior2.6',
 'bior2.8',
 'bior3.1',
 'bior3.3',
 'bior3.5',
 'bior3.7',
 'bior3.9',
 'bior4.4',
 'bior5.5',
 'bior6.8',
 'cgau1',
 'cgau2',
 'cgau3',
```

```
'cgau4',
'cgau5',
'cgau6',
'cgau7',
'cgau8',
'cmor',
'coif1',
'coif2',
'coif3',
'coif4',
'coif5',
'coif6',
'coif7',
'coif8',
'coif9',
'coif10',
'coif11',
'coif12',
'coif13',
'coif14',
'coif15',
'coif16',
'coif17',
'db1',
'db2',
'db3',
'db4',
'db5',
'db6',
'db7',
'db8',
'db9',
'db10',
'db11',
'db12',
'db13',
'db14',
'db15',
'db16',
'db17',
'db18',
'db19',
'db20',
'db21',
'db22',
'db23',
'db24',
```

```
'db25',
'db26',
'db27',
'db28',
'db29',
'db30',
'db31',
'db32',
'db33',
'db34',
'db35',
'db36',
'db37',
'db38',
'dmey',
'fbsp',
'gaus1',
'gaus2',
'gaus3',
'gaus4',
'gaus5',
'gaus6',
'gaus7',
'gaus8',
'haar',
'mexh',
'morl',
'rbio1.1',
'rbio1.3',
'rbio1.5',
'rbio2.2',
'rbio2.4',
'rbio2.6',
'rbio2.8',
'rbio3.1',
'rbio3.3',
'rbio3.5',
'rbio3.7',
'rbio3.9',
'rbio4.4',
'rbio5.5',
'rbio6.8',
'shan',
'sym2',
'sym3',
'sym4',
'sym5',
```

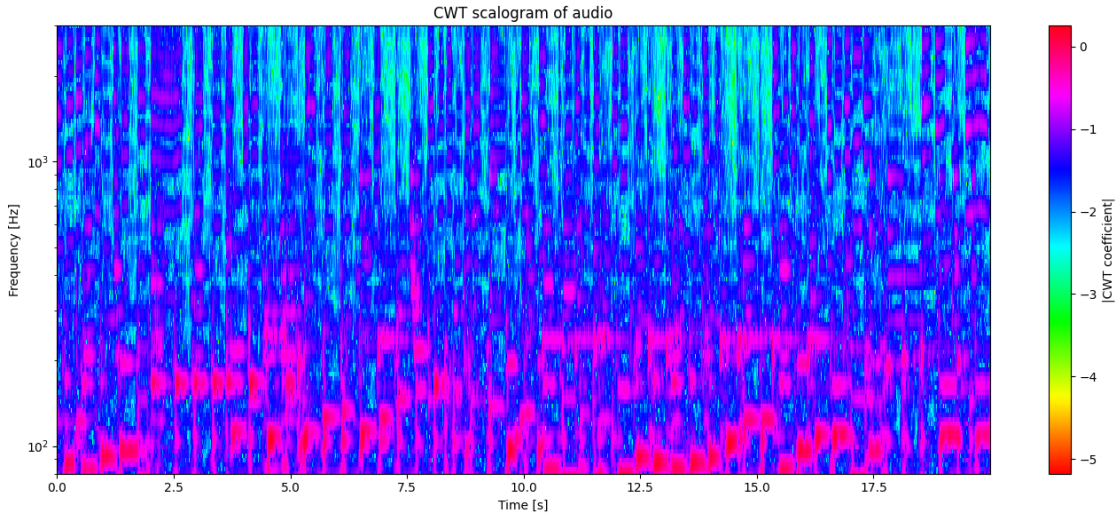
```
 'sym6',
 'sym7',
 'sym8',
 'sym9',
 'sym10',
 'sym11',
 'sym12',
 'sym13',
 'sym14',
 'sym15',
 'sym16',
 'sym17',
 'sym18',
 'sym19',
 'sym20']
```

[187]:
```python
# Write a Python function to compute the continuous wavelet transform (CWT) of␣
 ↪a given signal using the real Morlet wavelet.
import numpy as np
import matplotlib.pyplot as plt
import pywt

# Define the signal
t = np.linspace(0, 1, 1000, endpoint=False)
x = np.cos(2 * np.pi * 7 * t) + np.sin(2 * np.pi * 13 * t)

# Compute the CWT
scales = np.arange(1, 128)
coefficients, frequencies = pywt.cwt(x, scales, 'morl', 1/1000)

# Plot the CWT
plt.imshow(np.abs(coefficients), extent=[0, 1, 1, 128],  aspect='auto',␣
 ↪vmax=abs(coefficients).max(), vmin=-abs(coefficients).max())
plt.title('CWT with real Morlet')
plt.ylabel('Scale')
plt.xlabel('Time [sec]')
plt.show()
```



[188]:
```
# CWT of a sound file
from pylab import *
from scipy.io import wavfile
import pywt

fs, data = wavfile.read("02. School Boy-9.wav")

# stereo -> mono
x = data.mean(axis=1) if data.ndim == 2 else data
x = x.astype(float)
x = x / abs(x).max()

# first 20 seconds
x = x[:int(20 * fs)]

# simple downsampling
q = 5
x = x[::q]
fs = fs / q    # 8820 Hz
```

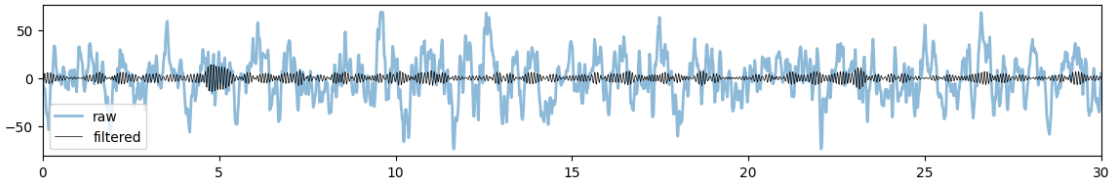
```
t = arange(len(x)) / fs

# CWT
wavelet = "cmor12.5-1.0"
freqs = geomspace(80, 3000, 100)

scales = pywt.central_frequency(wavelet) * fs / freqs

coef, freq_out = pywt.cwt(
    x,
    scales,
    wavelet,
    sampling_period=1/fs,
    method="fft"
)

power = abs(coef)
Z = log10(power + 1e-12)
# plot every few time samples
step = 5

# normalize each frequency row
#Z = Z - Z.mean(axis=1, keepdims=True)
#Z = Z / Z.std(axis=1, keepdims=True)

figure(figsize=(14, 6))
pcolormesh(t[::step], freq_out, Z[:, ::step], cmap=cm.hsv, shading="auto")
yscale("log")
ylim(80, 3000)
xlabel("Time [s]")
ylabel("Frequency [Hz]")
title("CWT scalogram of audio")
colorbar(label="|CWT coefficient|")
tight_layout()
show()
```

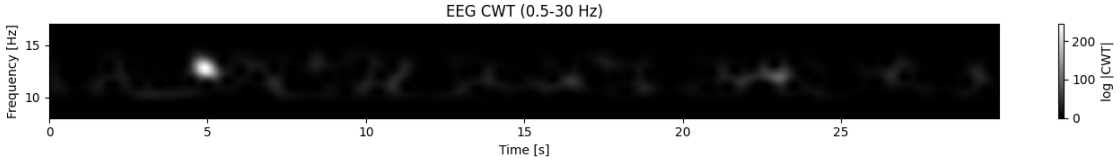


0.1.1 visualizing spindles in sleep EEG signals

[189]:
```
from pylab import *
import pywt

# EEG
fs = 200.0
x = fromfile("eeg.bin", dtype=float32)
x = x[10000:16000:2]
fs = fs/2
# remove DC
x = x - x.mean()

b, a = signal.butter(
    4,
    [10, 14],
    btype="bandpass",
    fs=fs
)

# zero-phase filtering
xo = x.copy()
x = signal.filtfilt(b, a, x)

# frequencies of interest
freqs = linspace(8, 17, 100)

wavelet = "cmor15.5-1.0"
```

```
scales = pywt.central_frequency(wavelet) * fs / freqs

coef, freq_out = pywt.cwt(
    x,
    scales,
    wavelet,
    sampling_period=1/fs,
    method="fft"
)

# log-amplitude
Z = (abs(coef) + 1e-12)**2

# optional: normalize each frequency band
#Z = Z - Z.mean(axis=1, keepdims=True)

t = arange(len(x)) / fs

figure(figsize=(14,2))
plot(t, xo, linewidth=2, alpha=0.5, label='raw')
plot(t, x, 'k-', linewidth=0.5, label='filtered')
xlim([0,30])
legend()

figure(figsize=(14,2))
pcolormesh(
    t,
    freq_out,
    Z,
    shading="auto",
    cmap="gray"
)
ylim(freqs[0], freqs[-1])
#yscale('log')
xlabel("Time [s]")
ylabel("Frequency [Hz]")
title("EEG CWT (0.5-30 Hz)")
colorbar(label="log |CWT|")
tight_layout()
show()
```

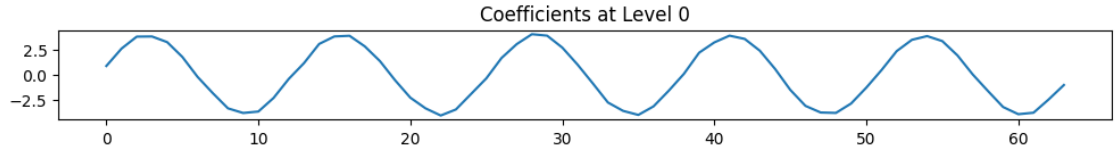


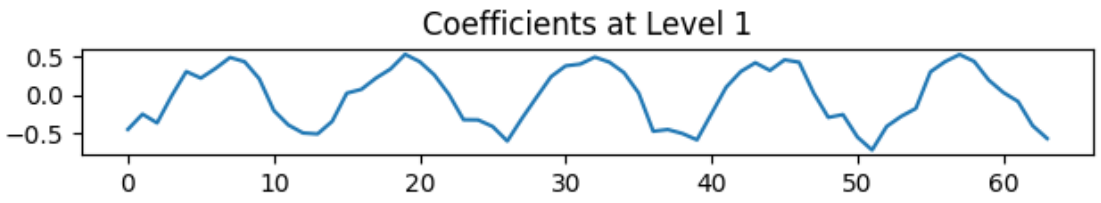


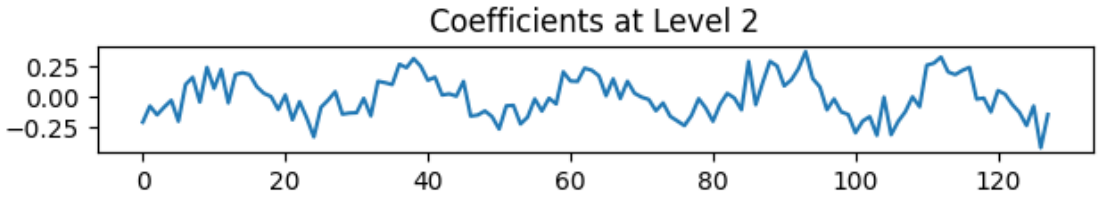
0.2 Discrete wavelets

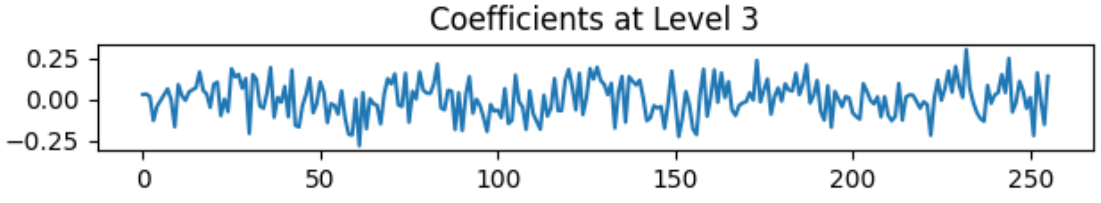
[190]:
```
import numpy as np
import pywt
import matplotlib.pyplot as plt

# Define the signal
t = np.linspace(0, 1, 1024)
x = np.sin(2 * np.pi * 5 * t) + np.random.randn(1024) * 0.1

# Compute the DWT using the Haar wavelet
coeffs = pywt.wavedec(x, 'haar', level=4)

plt.figure()

# Plot the approximation and detail coefficients
plt.figure(figsize=(10, 6))
for i, coeff in enumerate(coeffs):
    plt.subplot(len(coeffs), 1, i+1)
    plt.plot(coeff)
    plt.title(f'Coefficients at Level {i}')
    plt.tight_layout()
    plt.show()
```

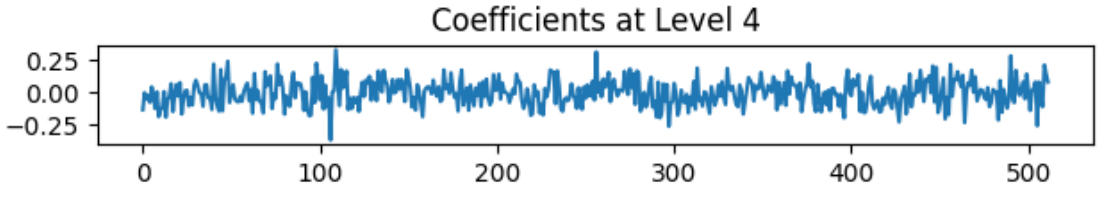
```
<Figure size 640x480 with 0 Axes>
```





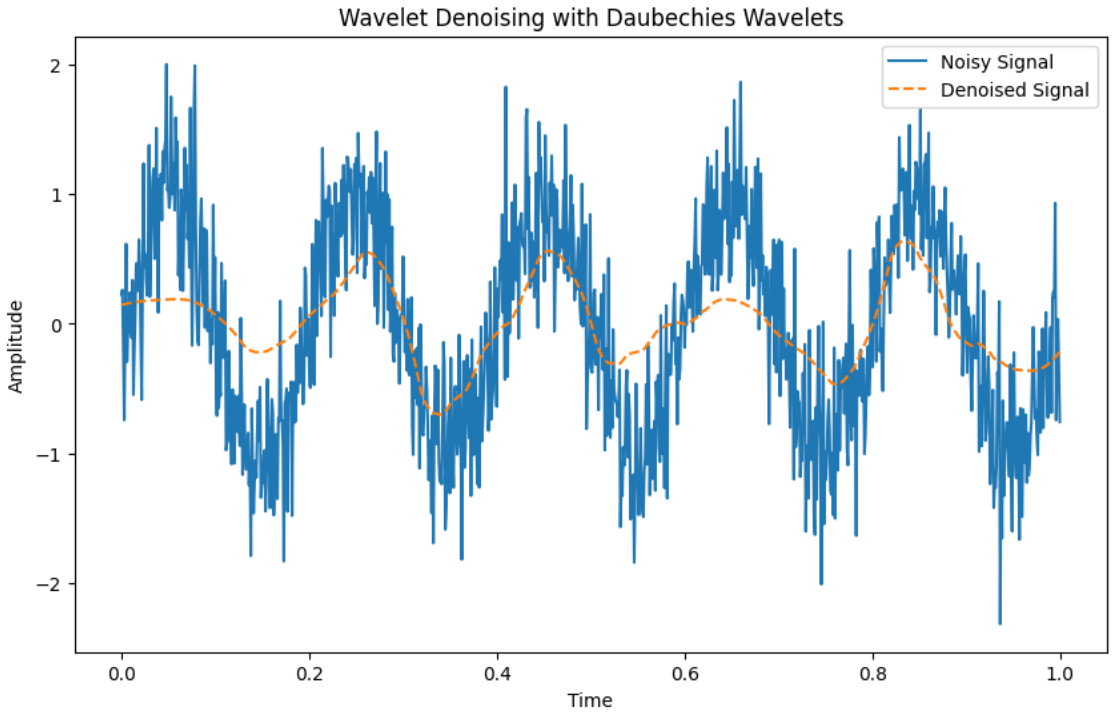






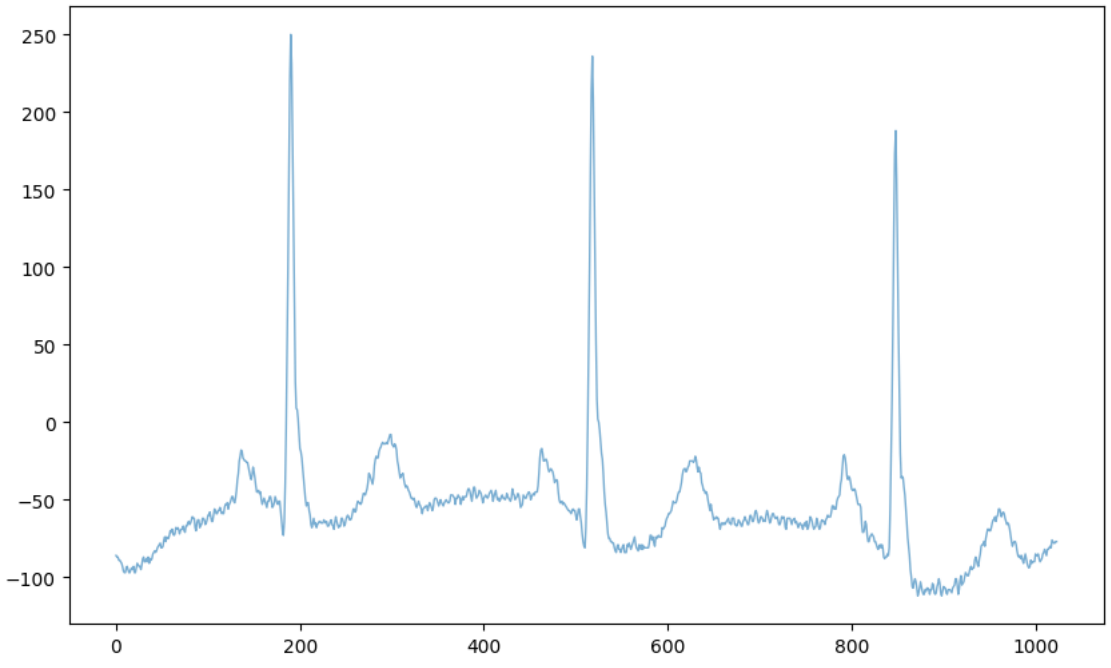
[191]:
```python
import numpy as np
import pywt
import matplotlib.pyplot as plt

# Generate a noisy signal
t = np.linspace(0, 1, 1024)
x = np.sin(2 * np.pi * 5 * t) + np.random.randn(1024) * 0.4

# Perform wavelet denoising using Daubechies wavelets
wavelet = 'db4'
```

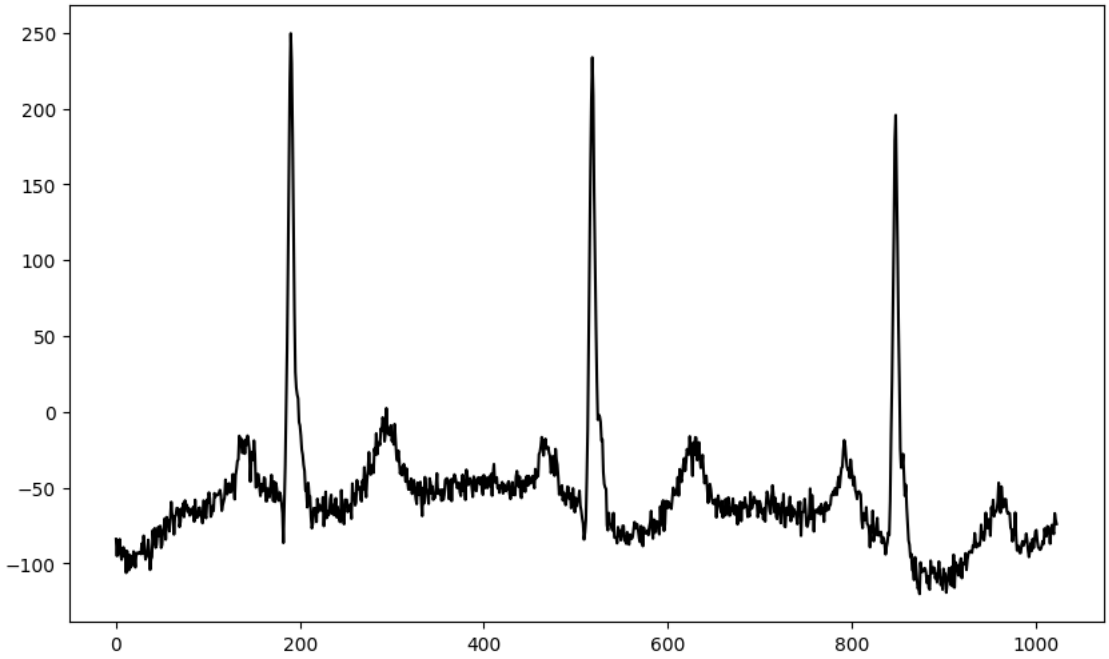
```
coeffs = pywt.wavedec(x, wavelet)
threshold = np.sqrt(2 * np.log(len(x)))
denoised_coeffs = map(lambda x: pywt.threshold(x, threshold, mode='soft'),␣
 ↪coeffs)
denoised_signal = pywt.waverec(list(denoised_coeffs), wavelet)

# Plot the original and denoised signals
plt.figure(figsize=(10, 6))
plt.plot(t, x, label='Noisy Signal')
plt.plot(t, denoised_signal, label='Denoised Signal', linestyle='--')
plt.legend()
plt.title('Wavelet Denoising with Daubechies Wavelets')
plt.xlabel('Time')
plt.ylabel('Amplitude')
plt.show()
```

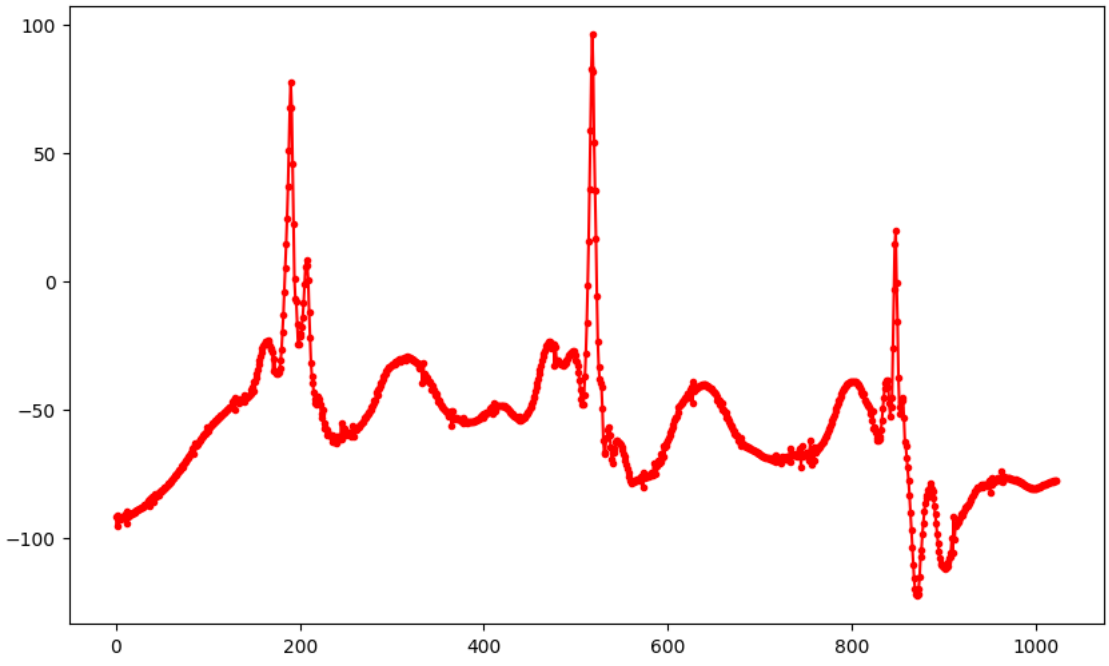


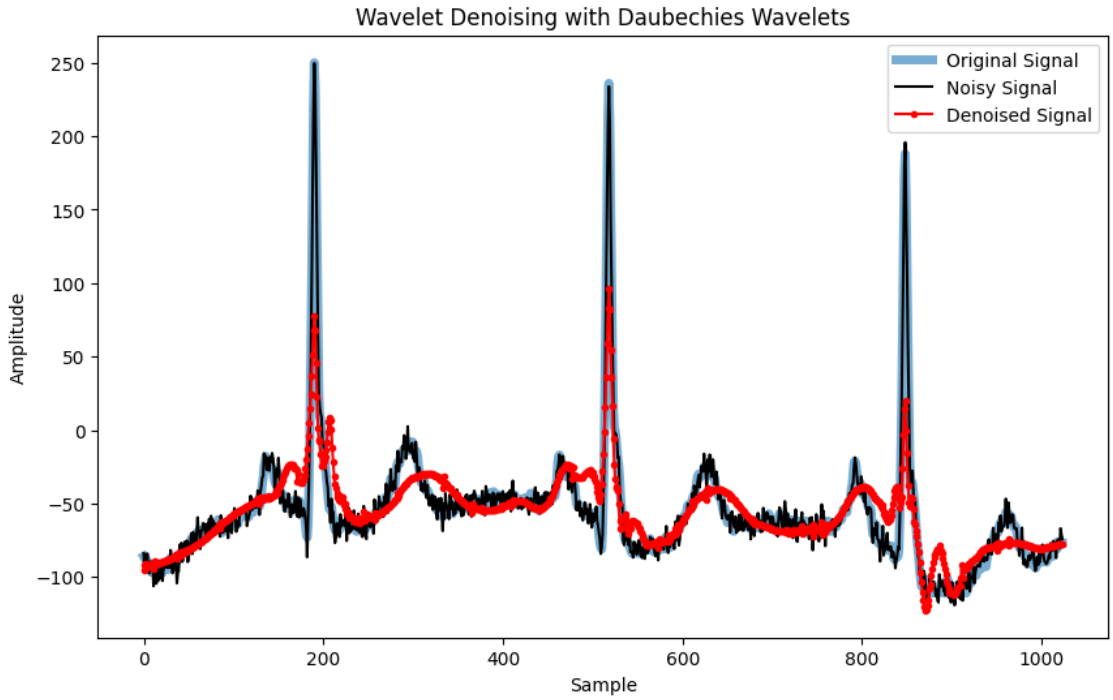
[192]:
```
# Create a Python script to demonstrate the effect of vanishing moments in␣
 ↪wavelets by denoising a signal.
import numpy as np
import matplotlib.pyplot as plt
import pywt
import pywt.data
```

```python
# Load a noisy signal
original = pywt.data.ecg()
noisy = original + 5.1 * np.random.randn(len(original))

# Wavelet denoising using Daubechies wavelets with different vanishing moments
wavelet = 'db6'
coeffs = pywt.wavedec(noisy, wavelet)
threshold = 0.5
coeffs[1:] = [pywt.threshold(i, value=threshold*max(i)) for i in coeffs[1:]]
reconstructed = pywt.waverec(coeffs, wavelet)

# Plot
plt.figure(figsize=(10, 6))
plt.plot(original, label='Original Signal', linewidth=1, alpha=0.6)
plt.figure(figsize=(10, 6))
plt.plot(noisy, 'k-', label='Noisy Signal')
plt.figure(figsize=(10, 6))
plt.plot(reconstructed, 'r.-', label='Denoised Signal')

plt.figure(figsize=(10, 6))
plt.plot(original, label='Original Signal', linewidth=5, alpha=0.6)
plt.plot(noisy, 'k-', label='Noisy Signal')
plt.plot(reconstructed, 'r.-', label='Denoised Signal')
plt.legend()
plt.title('Wavelet Denoising with Daubechies Wavelets')
plt.xlabel('Sample')
plt.ylabel('Amplitude')
plt.show()
```




250
200
150
100
50
0
−50
−100
0
200
400
600
800
1000

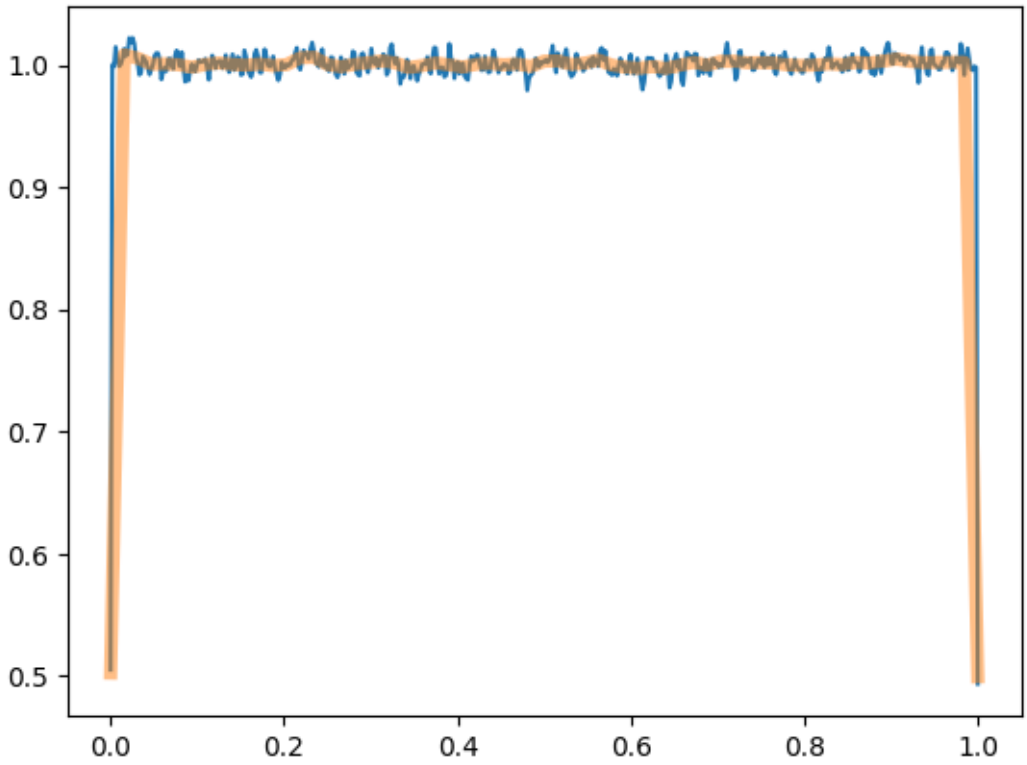

100
50
0
−50
−100
0
200
400
600
800
1000



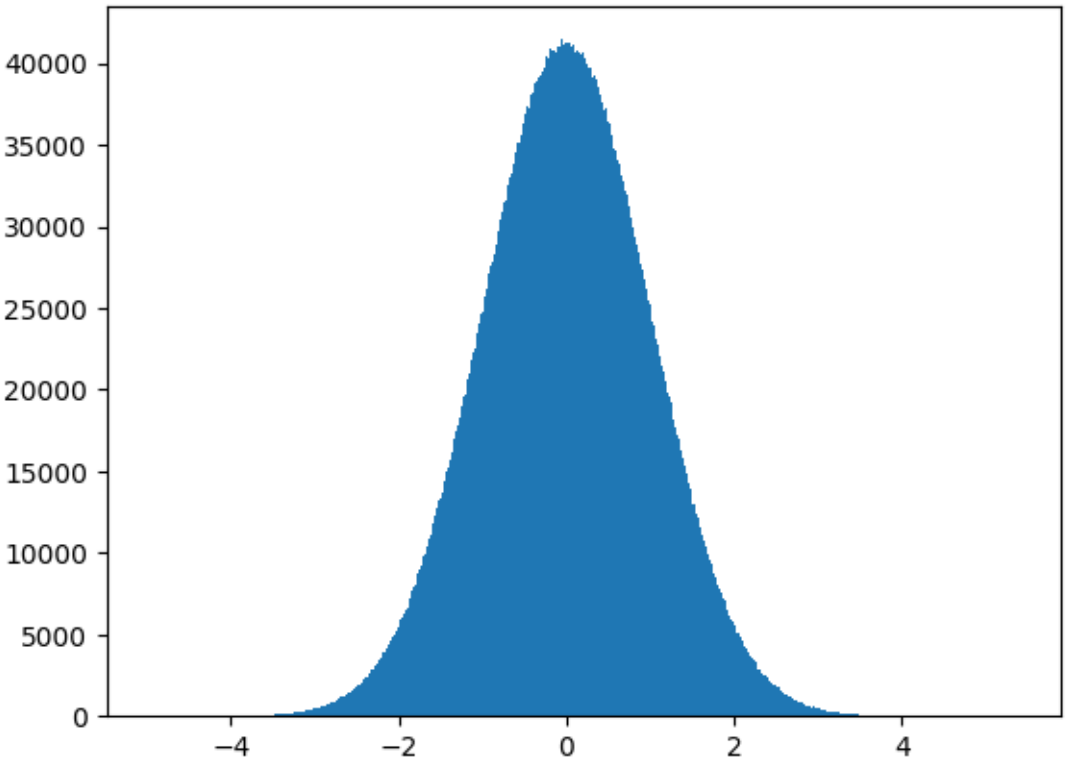
0.3 Colored noise generation

[194]:
```
############################
# Generating 'coloured' noise
############################
# white noise
from  numpy import random
r = random.randn(10000000) # normally distributed random numbers
r = r- r.mean()
# check the flat PSD
s, f = mlab.psd(r, Fs=2, NFFT=1024, noverlap=512)
plot(f, s)
s, f = mlab.psd(r, Fs=2, NFFT=128, noverlap=100) # the same as welch
plot(f, s, linewidth=5, alpha=0.5)
```

[194]:
```
[<matplotlib.lines.Line2D at 0x713265746c30>]
```

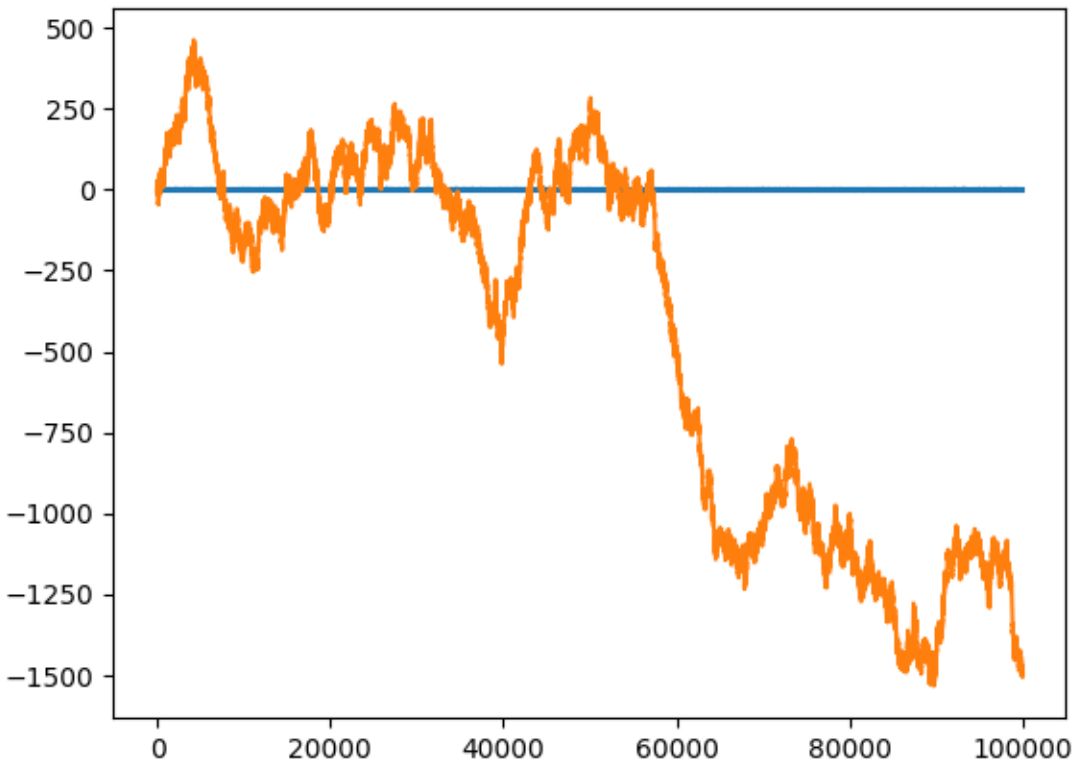


[195]:
```
h = hist(r, bins=1000)
```



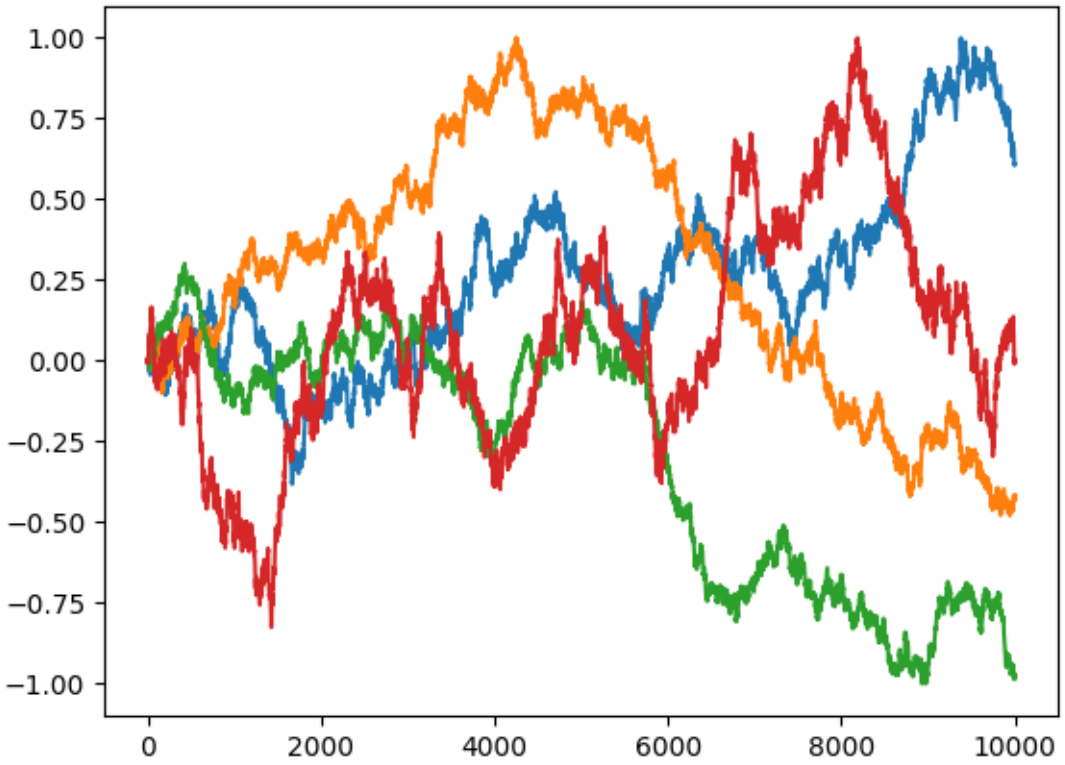
[196]:
```
# brown noise
cr = r.cumsum()
plot(r[:1000000:10])
plot(cr[:1000000:10])
```

[196]: [<matplotlib.lines.Line2D at 0x713265a9a7e0>]

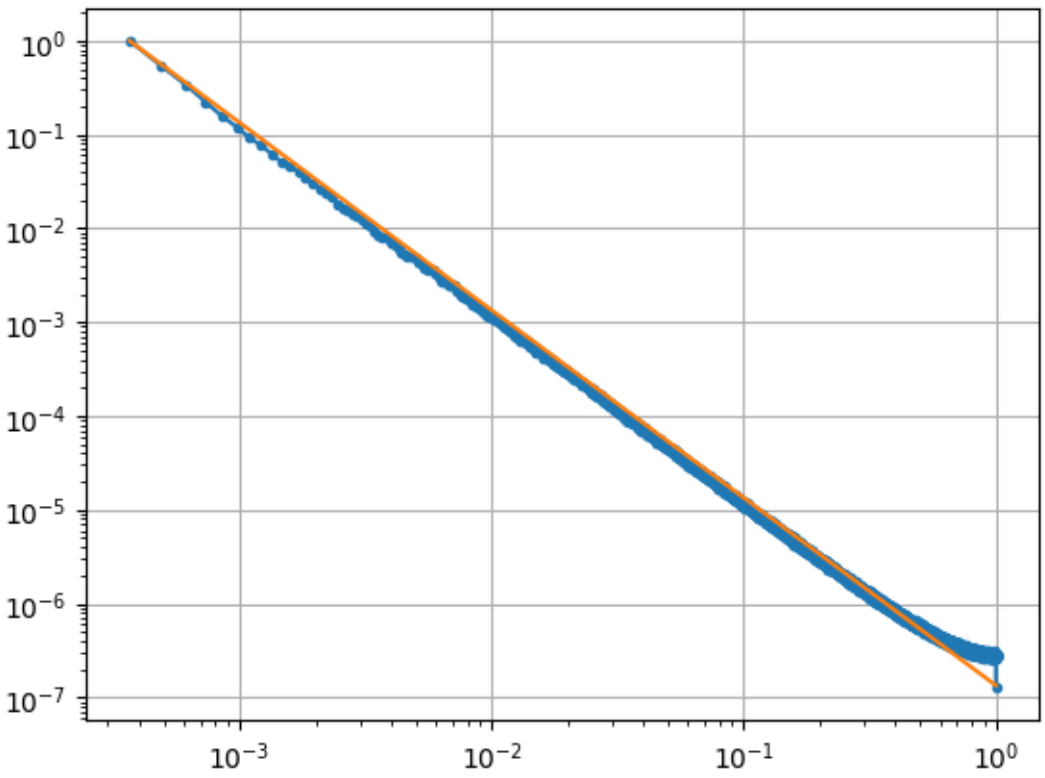


[197]:
```
# demonstrate the self-similarity of the brownian noise by changing the range
# at all ranges the behavior is similar (can't tell from the image which is the␣
 ↪one with 10000 and which the one with 1000000 points)
plot(cr[:10000]/abs(cr[:10000]).max())
plot(cr[:100000:10]/abs(cr[:100000:10]).max())
plot(cr[:1000000:100]/abs(cr[:1000000:100]).max())
plot(cr[:10000000:1000]/abs(cr[:10000000:1000]).max())
```

[197]:
```
[<matplotlib.lines.Line2D at 0x713265a66fc0>]
```

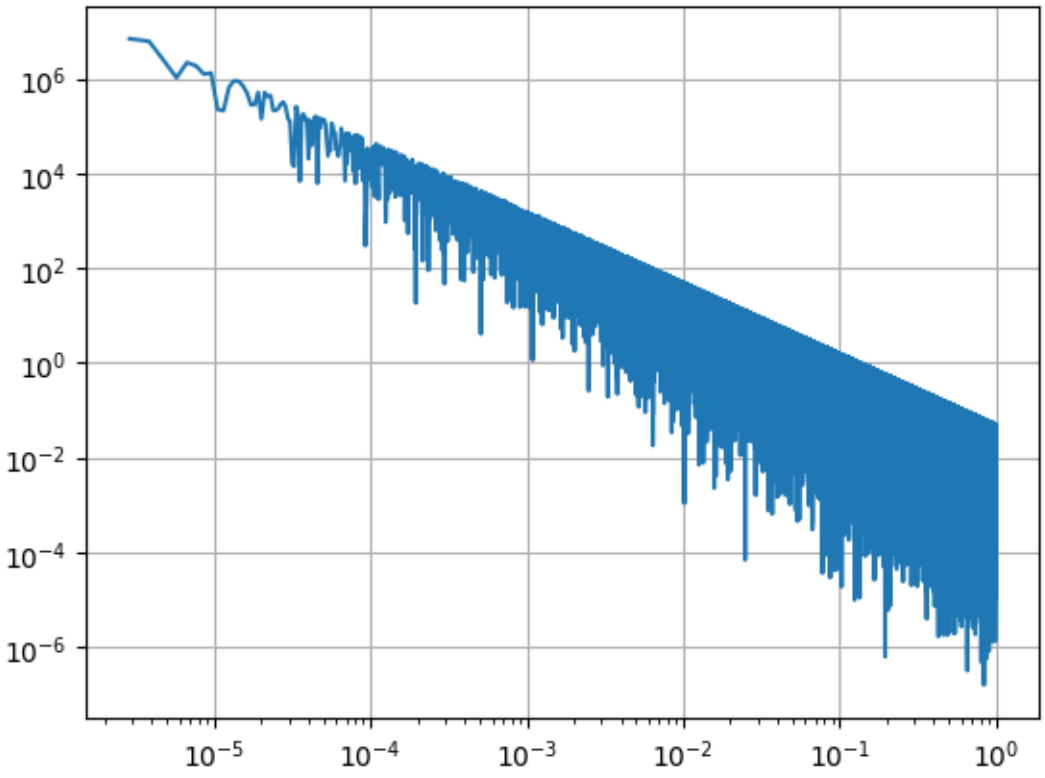


[198]:
```
# check the ~1/f^2 PSD
cs, f = mlab.psd(cr, Fs=2, NFFT=2**14, noverlap=2**13)
loglog(f[3:], cs[3:]/cs[3], '.-')
loglog(f[3:], 1/(f[3:]/f[3])**2)
grid()
```



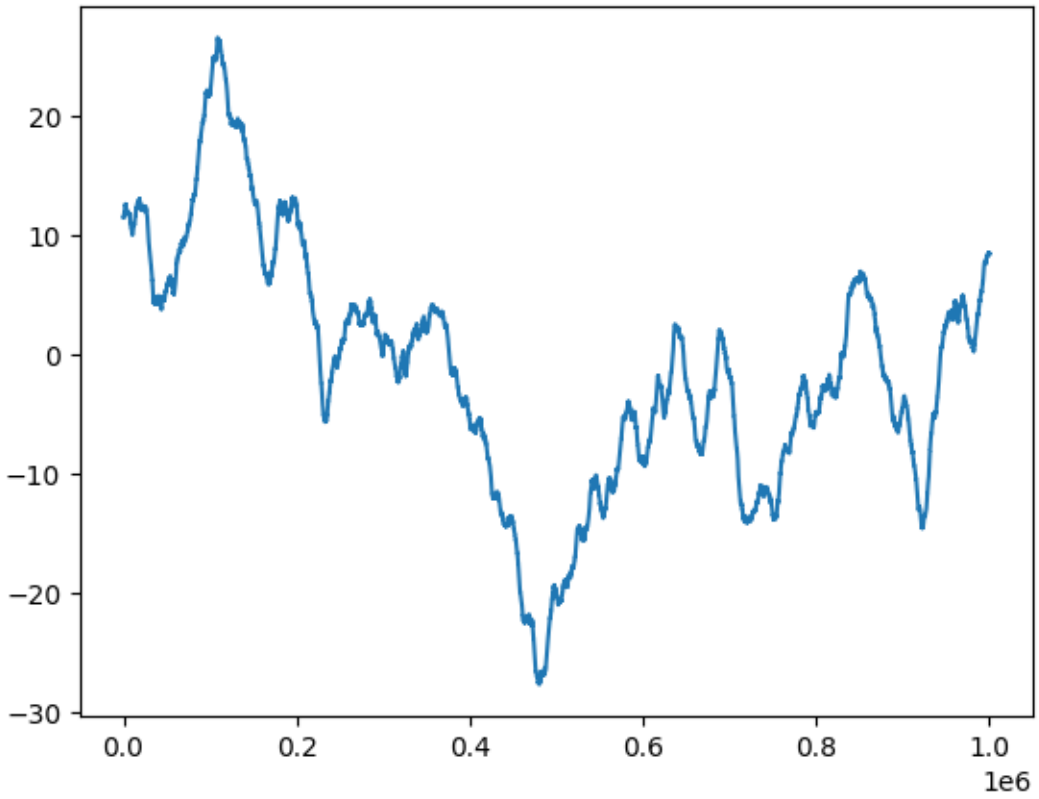
[199]:
```
# any coloured noise by inverse Fourier transform
f = linspace(0, 1, 2**20)+ 1e-8 # generate frequencies shifted so that we don't␣
 ↪have problem in zero
a = f**(-1.5)  # amplitude is a power function of frequency -> PSD(f)~f**(-3.0)
a *= random.rand(len(a))*0.05 # add some noise over the smooth dependence
a = a[3:] # avoid the divergence at 0Hz
f = f[3:]
loglog(f, a)

grid()
print(a)
```

```
[6.93395803e+06 6.13537850e+06 2.35473835e+06 … 1.12309564e-02
 3.52942859e-02 4.28969436e-02]
```

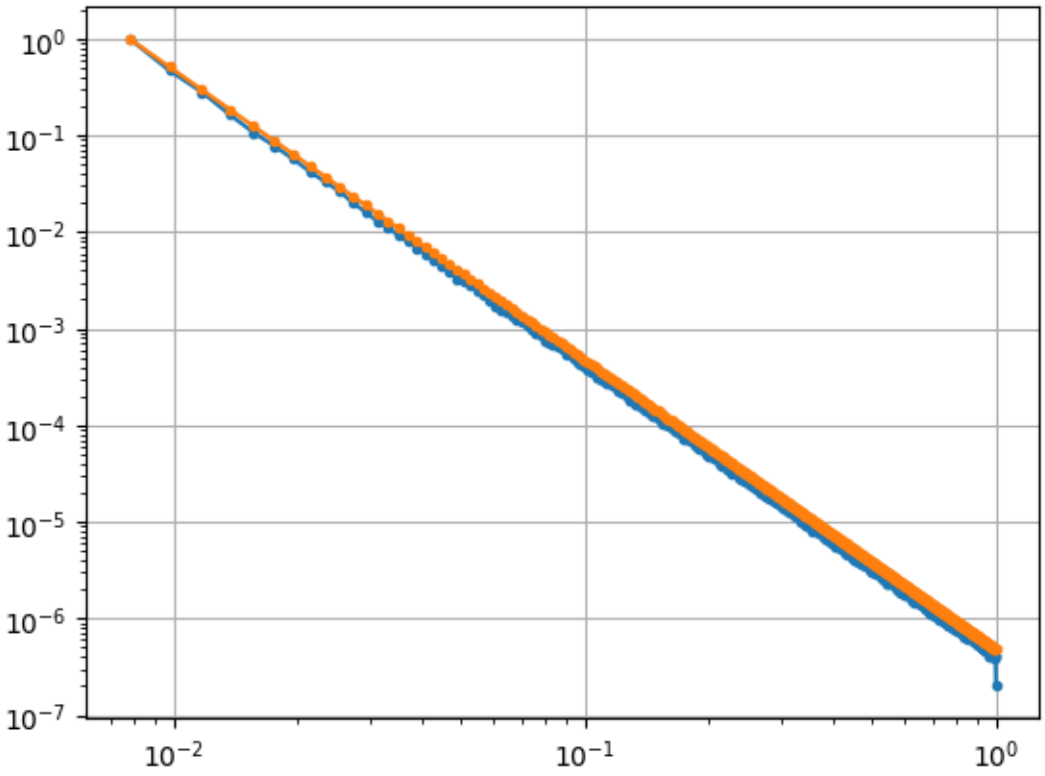


[200]:
```
X = a*exp(2*pi*1j*random.rand(len(a))) # multiply with random phase factor
noise = irfft(X[:len(X)//2+1])         # get the signal (coloured noise)
plot(noise[:1000000])
```

[200]: [<matplotlib.lines.Line2D at 0x71327367e600>]



[201]:
```
s, f = mlab.psd(noise, Fs=2, NFFT=2**10) # check the PSD
loglog(f[4:], s[4:]/s[4], '.-')
loglog(f[4:], (f[4:]/f[4])**(-3), '.-')
grid()
```



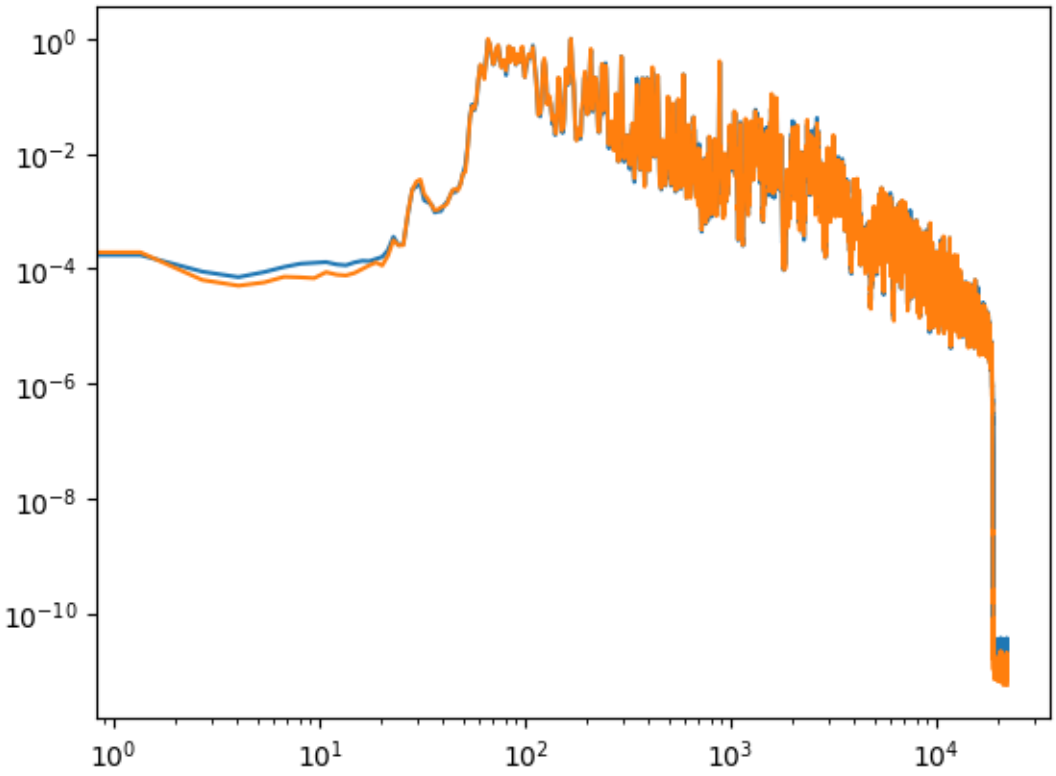
0.4 Surrogate data

[202]:
```
# create a surrogate sound file with the same spectral density as the original
from scipy.io import wavfile
from pylab import *
from scipy import signal
from numpy import fft
from numpy import random
fs, data = wavfile.read("02. School Boy-9.wav")
x = data[:,0]
dt = 1/fs
X = dt*fft.fft((x))
print(X)
xx = fs*fft.ifft(X)
X_surr = abs(X)*exp(1j*2*pi*random.rand(len(xx)))
x_surr = irfft(X_surr[:len(X)//2+1])
x_surr
```

```
[-4.55646259 +0.j          21.75253119 +0.91056594j
 -5.62082715-13.08794847j …  3.47453888-27.7174622j
 -5.62082715+13.08794847j 21.75253119 -0.91056594j]
```

[202]:
```
array([-0.01983952, -0.07333472, -0.07930347, …,  0.03108834,
        0.01972357,  0.02311509])
```

[203]:
```
f, sy = signal.welch(
    x_surr,
    fs=fs,
    window="hann",
    nperseg=2**15,
    noverlap=2**14,
    detrend="constant",
    scaling="density"
)
plt.loglog(f, sy/sy.max())
plt.grid()
f, sx = signal.welch(
    x,
    fs=fs,
    window="hann",
    nperseg=2**15,
    noverlap=2**14,
    detrend="constant",
    scaling="density"
)
plt.loglog(f, sx/sx.max())
plt.grid()
```



[204]:
```
x_surr = int16(32767*x_surr/abs(x_surr).max())

data_surr = column_stack([x_surr, x_surr])
wavfile.write("surr.wav", fs, data_surr)
```

[ ]:
```

```